GETTING STARTED WITH

DSA: Arrays & Strings, Time & Space Complexity, Linked Lists, Stack & Queue

A Beginner-Friendly Reference Guide for Developers

CONTENTS

— Introduction To Data Structures And Algorithms
— Arrays: The Foundation Of Linear Data Storage
— Strings: Manipulation, Patterns, And Techniques
— Time And Space Complexity: Measuring Algorithm Efficiency
— Linked Lists: Dynamic And Flexible Data Storage
— Stacks And Queues: Lifo And Fifo Data Structures
— Conclusion

A Skilling Refcard provides concise reference guides on key technology topics for practitioners.

WRITTEN BY

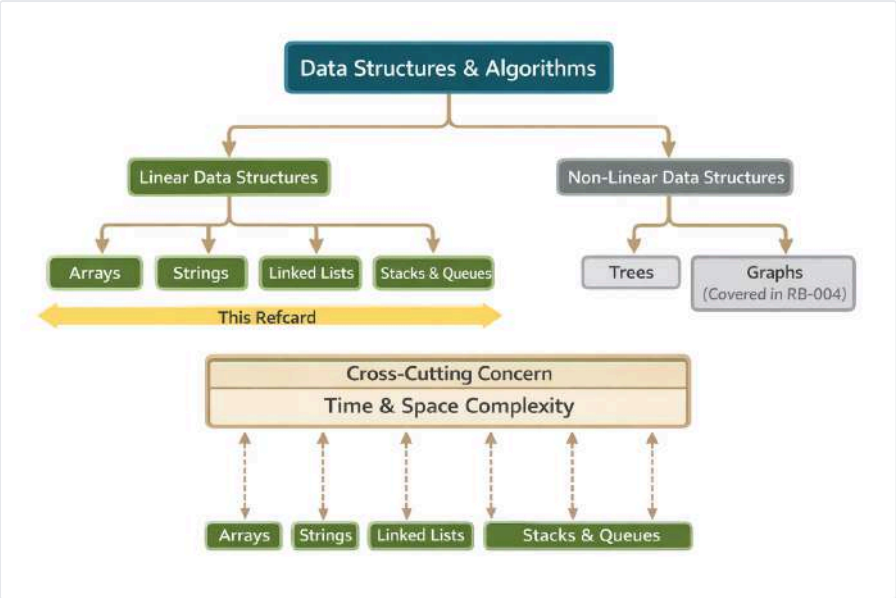
KMIT Group- FS Team

REFCARD

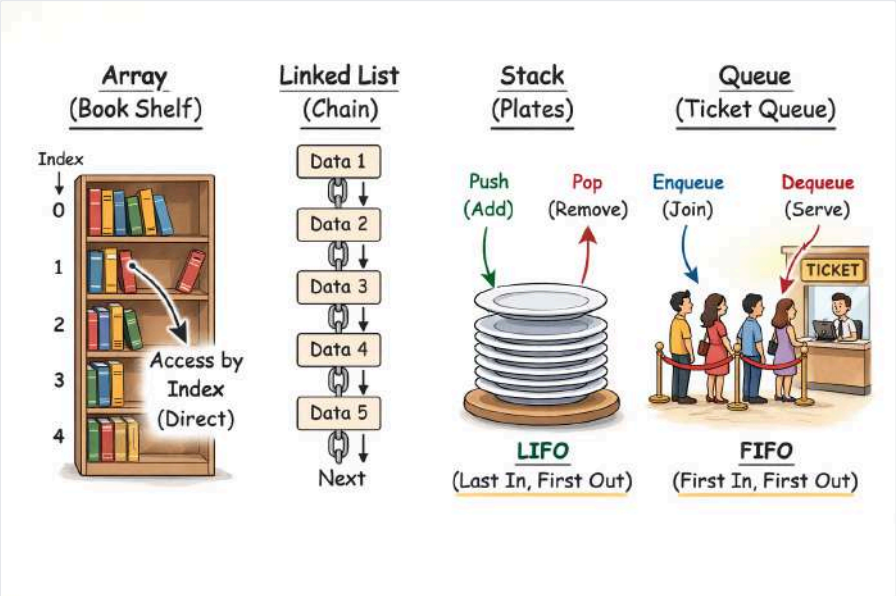
RB-C3-2026-DSA

INTRODUCTION TO DATA STRUCTURES AND ALGORITHMS

Data Structures and Algorithms (DSA) form the backbone of efficient software development. Understanding how data is organized in memory and how algorithms manipulate that data is critical for writing performant code, cracking technical interviews, and building scalable systems. This section lays the foundation by explaining what data structures are, why algorithms matter, and how choosing the right combination can make or break your application's performance.



DSA Landscape Overview



WHAT IS A DATA STRUCTURE?

A data structure is a specialized format for organizing, storing, and managing data in a computer's memory so that it can be accessed and modified efficiently. Think of it like choosing the right container for different items: you wouldn't store soup in a paper bag or carry coins in a swimming pool. Similarly, storing a list of student grades in an array is efficient for random access, but if you frequently insert and delete students, a linked list might be better. Every data structure has trade-offs in terms of memory usage, access speed, and ease of modification. Choosing wisely is the hallmark of a good developer.

WHAT IS AN ALGORITHM?

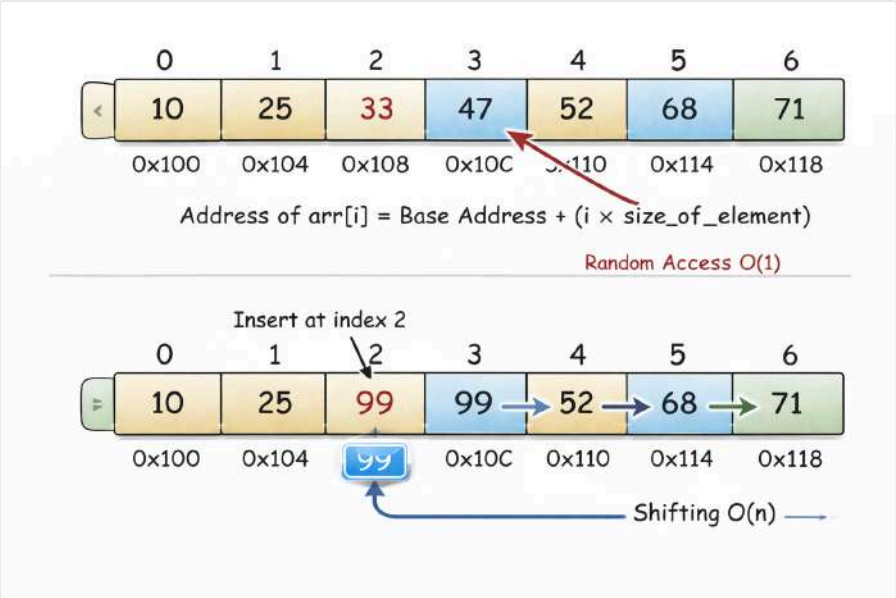
An algorithm is a step-by-step procedure or formula for solving a problem. Consider a real-world scenario: you want to find a contact named 'Ravi' in your phone. You could scroll from top to bottom (linear search), or you could jump to the 'R' section directly (a form of indexed search). Both approaches find Ravi, but one is dramatically faster. In programming, algorithms define the logic for searching, sorting, inserting, and deleting data within structures. The efficiency of your algorithm directly impacts how fast your application responds, especially when handling millions of records like an e-commerce product catalog or a social media feed.

WHY DSA MATTERS FOR DEVELOPERS

Imagine you are building a food delivery app like Swiggy or Zomato. When a user searches for 'biryani', your app must search through thousands of restaurant menus and return results in milliseconds. A poorly chosen data structure or algorithm might cause a 5-second delay, frustrating users and losing business. DSA knowledge helps you write code that scales from 100 users to 10 million users without collapsing. It is also the number one topic tested in technical interviews at companies like Google, Amazon, Microsoft, and top startups. Mastering DSA is not optional — it is the price of entry into professional software development.

ARRAYS: THE FOUNDATION OF LINEAR DATA STORAGE

Arrays are the simplest and most widely used data structure in programming. They store elements of the same type in contiguous memory locations, enabling fast access by index. Whether you are storing sensor readings from an IoT device, pixel values in an image, or player scores in a leaderboard, arrays are likely your first tool of choice.



Array Memory Layout

Array Operations Time Complexity

OPERATION	BEST CASE	AVERAGE CASE	WORST CASE
Access by Index	O(1)	O(1)	O(1)
Search (Unsorted)	O(1)	O(n)	O(n)
Search (Sorted - Binary)	O(1)	O(log n)	O(log n)
Insert at End	O(1)	O(1)	O(1)
Insert at Beginning	O(n)	O(n)	O(n)
Insert at Middle	O(1)	O(n)	O(n)
Delete from End	O(1)	O(1)	O(1)
Delete from Beginning	O(n)	O(n)	O(n)

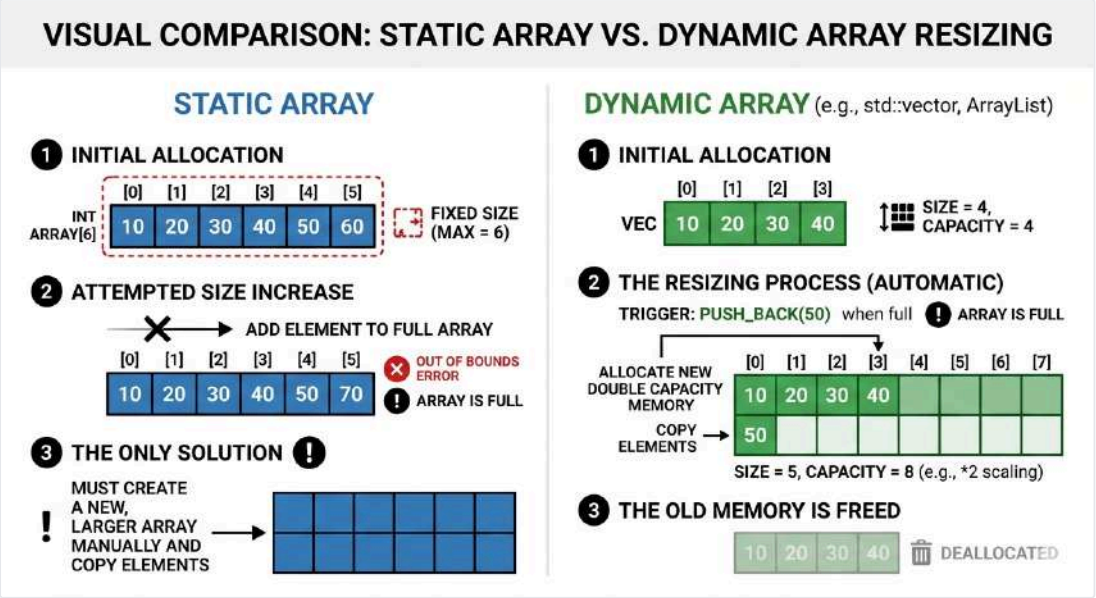
JAVA — Array Declaration, Traversal, and Common Operations in Java

```java
// Scenario: Managing daily temperatures for a weather app
public class ArrayBasics {
    public static void main(String[] args) {
        // Declaration and initialization
        int[] temps = {32, 35, 28, 40, 38, 29, 33};

        // Access by index - O(1)
        System.out.println("Wednesday temp: " + temps[2]); // 28

        // Traversal - O(n)
        int sum = 0;
        for (int t : temps) {
            sum += t;
        }
        double avg = (double) sum / temps.length;
        System.out.println("Average: " + avg);

        // Find maximum temperature - O(n)
        int max = temps[0];
        for (int i = 1; i < temps.length; i++) {
            if (temps[i] > max) max = temps[i];
        }
        System.out.println("Hottest day: " + max + "°C");
```

```
        // Linear search: find first day above 35 - O(n)
        for (int i = 0; i < temps.length; i++) {
            if (temps[i] > 35) {
                System.out.println("First hot day: index " + i);
                break;
            }
        }
    }
}
```

JAVA Two-Pointer Technique: Finding a Pair with Target Sum in a Sorted Array

```
// Scenario: E-commerce — find two items whose prices sum to a gift card value
public class TwoPointerPairSum {
    public static int[] findPair(int[] prices, int giftCard) {
        int left = 0, right = prices.length - 1;
        while (left < right) {
            int sum = prices[left] + prices[right];
            if (sum == giftCard) {
                return new int[]{left, right};
            } else if (sum < giftCard) {
                left++;   // Need a larger sum
            } else {
                right--;  // Need a smaller sum
            }
        }
        return new int[]{-1, -1}; // No pair found
    }

    public static void main(String[] args) {
        int[] prices = {100, 200, 350, 500, 700, 900}; // sorted
        int giftCard = 850;
        int[] result = findPair(prices, giftCard);
        System.out.println("Items at index " + result[0]
            + " (₹" + prices[result[0]] + ") and index "
            + result[1] + " (₹" + prices[result[1]] + ")");
        // Output: Items at index 1 (₹200) and index 4 (₹700)
    }
}
```



STATIC VS DYNAMIC ARRAYS

A static array has a fixed size determined at creation time. Once you declare int[] arr = new int[5], you cannot add a 6th element without creating a new, larger array. In contrast, dynamic arrays (like Java's ArrayList or Python's list) automatically resize when full. They typically double their capacity and copy existing elements to the new memory block. This resizing costs O(n) occasionally, but since it happens infrequently, the amortized cost of adding an element remains O(1). Real-world scenario: a chat application storing messages uses a dynamic array because the number of messages is unpredictable.

COMMON ARRAY PATTERNS: SLIDING WINDOW

The sliding window pattern is used when you need to process contiguous subarrays of a fixed or variable size. Imagine a stock trading app that needs to find the maximum profit over any 7-day window in a 365-day dataset. Instead of recalculating the sum for every window from scratch (O(n×k)), you slide the window by subtracting the element leaving the window and adding the new element entering it, achieving O(n) time. This pattern appears in problems like maximum subarray sum, longest substring without repeating characters, and finding averages of subarrays.

JAVA — Sliding Window: Maximum sum of k consecutive elements

```java
// Scenario: Find the best 3-day sales period
public static int maxSumWindow(int[] sales, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += sales[i];
    int maxSum = windowSum;
    for (int i = k; i < sales.length; i++) {
        windowSum += sales[i] - sales[i - k];
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
// sales = {200, 150, 300, 250, 400, 100, 350}
// k=3 → Best 3-day window: [250, 400, 100]? No → [300, 250, 400] = 950
```

MULTI-DIMENSIONAL ARRAYS

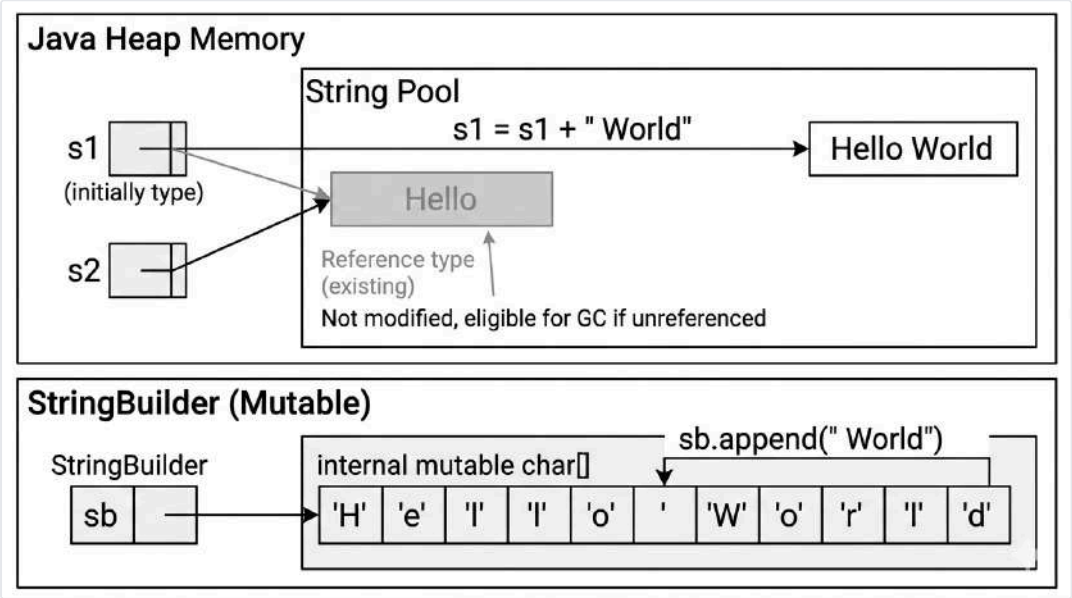
A 2D array is essentially an array of arrays, forming a grid or matrix structure. This is used extensively in real-world applications: a digital image is a 2D array of pixel values, a spreadsheet is a 2D array of cells, and a chess board is an 8×8 grid. Accessing element matrix[i][j] takes O(1) time. Traversing the entire m×n matrix takes O(m×n) time. Common operations include row-wise and column-wise traversal, diagonal traversal, matrix rotation, and spiral order traversal. In game development, a 2D array can represent a tile map where each cell stores terrain type.

A 4×4 grid showing a tic-tac-toe style layout. Rows are labeled 0-3 on the left, columns labeled 0-3 on top. Cell [1][2] is highlighted with a yellow background. Row-major order traversal is shown with a snake-like arrow going left-to-right in row 0, then left-to-right in row 1, etc. The formula 'Address = Base + (i × cols + j) × size' is written below.

2D Array as a Grid

STRINGS: MANIPULATION, PATTERNS, AND TECHNIQUES

Strings are sequences of characters and one of the most frequently used data types in software development. From validating user input in a registration form to parsing JSON API responses, string operations are everywhere. Understanding string immutability, encoding, and efficient manipulation techniques is essential for writing robust applications.



String Immutability in Java

String vs StringBuilder vs StringBuffer in Java

FEATURE	STRING	STRINGBUILDER	STRINGBUFFER
Mutability	Immutable	Mutable	Mutable
Thread Safety	Yes (immutable)	No	Yes (synchronized)
Performance	Slow for concatenation	Fast	Moderate
Use Case	Constants, keys	Single-threaded building	Multi-threaded building
Memory	Creates new object each time	Modifies in-place	Modifies in-place

JAVA — Common String Operations and Interview Patterns

```java
// Scenario: User input validation for a signup form
public class StringOps {

    // 1. Reverse a string (interview classic)
    public static String reverse(String s) {
        return new StringBuilder(s).reverse().toString();
    }

    // 2. Check if a string is a palindrome
    public static boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) return false;
            left++;
            right--;
        }
        return true;
    }

    // 3. Check if two strings are anagrams
    public static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) return false;
        int[] freq = new int[26];
        for (int i = 0; i < a.length(); i++) {
            freq[a.charAt(i) - 'a']++;
            freq[b.charAt(i) - 'a']--;
        }
        for (int count : freq) {
            if (count != 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(reverse("hello"));           // olleh
        System.out.println(isPalindrome("racecar"));    // true
        System.out.println(isAnagram("listen", "silent")); // true
    }
}
```

STRING IMMUTABILITY AND MEMORY IMPLICATIONS

In Java, strings are immutable — once created, their content cannot be changed. Every operation like concatenation or toUpperCase() creates a new String object in memory. This is safe for multi-threaded environments but costly for performance. If you concatenate strings inside a loop 10,000 times, you create 10,000 temporary objects. This is why StringBuilder exists: it maintains a mutable character buffer internally and only creates the final String when you call toString(). Real scenario: building an HTML email body by appending user data, headers, and footers — always use StringBuilder.

CHARACTER ENCODING: ASCII AND UNICODE

Computers store characters as numbers. ASCII uses 7 bits to represent 128 characters (A=65, a=97, 0=48). Unicode extends this to over 143,000 characters covering every language, emoji, and symbol. Java uses UTF-16 internally, where each char is 2 bytes. This matters in real-world apps: a messaging app supporting Hindi, Arabic, and Japanese must handle Unicode correctly. The common trick of using int[26] for frequency counting only works for lowercase English letters. For Unicode strings, use a HashMap<Character, Integer> instead. Understanding encoding prevents bugs like garbled text in internationalized applications.

PATTERN: LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS

This classic problem uses the sliding window technique with a HashSet. Imagine you are building an autocomplete feature and need to find the longest unique character sequence in a search query. You maintain a window with two pointers: expand the right pointer to include new characters, and shrink from the left when a

duplicate is found. The HashSet tracks characters in the current window. This achieves O(n) time complexity versus the brute-force O(n^3) approach. For the string 'abcabcbb', the answer is 3 ('abc'). This technique also applies to finding longest unique user sessions and DNA sequence analysis.

JAVA — Longest Substring Without Repeating Characters

```java
public static int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        while (window.contains(s.charAt(right))) {
            window.remove(s.charAt(left));
            left++;
        }
        window.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
// Input: "abcabcbb" → Output: 3 ("abc")
// Input: "pwwkew"   → Output: 3 ("wke")
```

STRING COMPRESSION AND RUN-LENGTH ENCODING

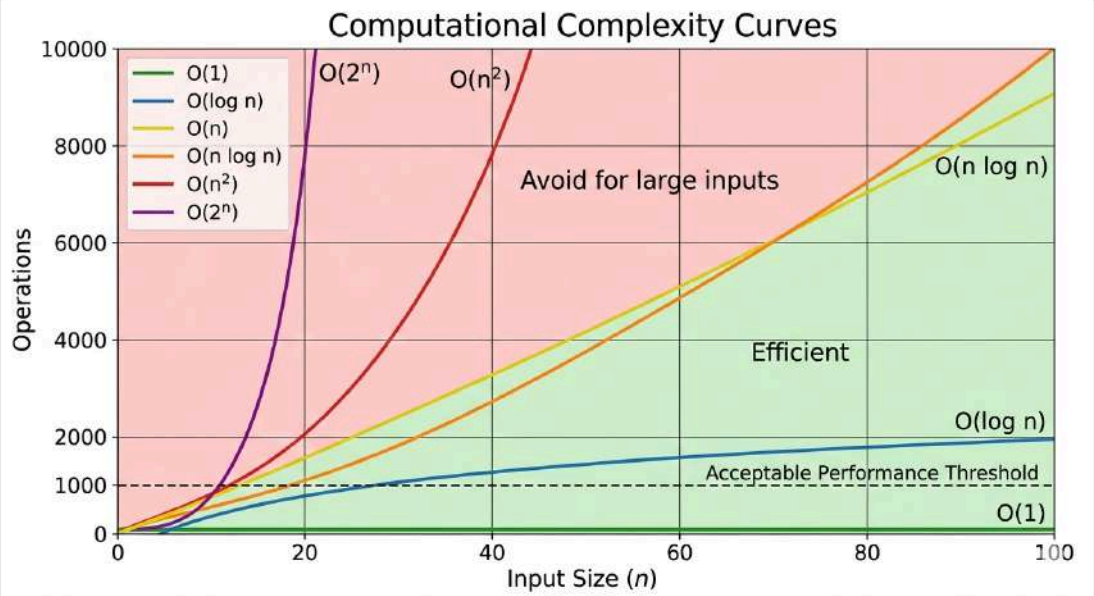
String compression reduces the size of repetitive strings by replacing consecutive duplicate characters with the character followed by its count. For example, 'aabcccccaaa' becomes 'a2b1c5a3'. This technique is used in data compression, file storage, and network transmission. In interviews, you are typically asked to return the compressed string only if it is shorter than the original. You iterate through the string, count consecutive characters using a pointer, and build the result with StringBuilder. Edge cases include single-character strings and strings with no repeats (where compression would actually increase length).

JAVA — Run-Length Encoding Implementation

```java
public static String compress(String s) {
    StringBuilder sb = new StringBuilder();
    int i = 0;
    while (i < s.length()) {
        char current = s.charAt(i);
        int count = 0;
        while (i < s.length() && s.charAt(i) == current) {
            count++;
            i++;
        }
        sb.append(current).append(count);
    }
    return sb.length() < s.length() ? sb.toString() : s;
}
// Input: "aabcccccaaa" → Output: "a2b1c5a3"
// Input: "abcdef"      → Output: "abcdef" (no compression benefit)
```

TIME AND SPACE COMPLEXITY: MEASURING ALGORITHM EFFICIENCY

Writing code that works is not enough — it must work efficiently. Time complexity measures how the runtime of an algorithm grows as input size increases, while space complexity measures additional memory consumed. Understanding Big O notation is the universal language developers use to compare algorithms and predict performance at scale.



Big O Complexity Growth Comparison Chart

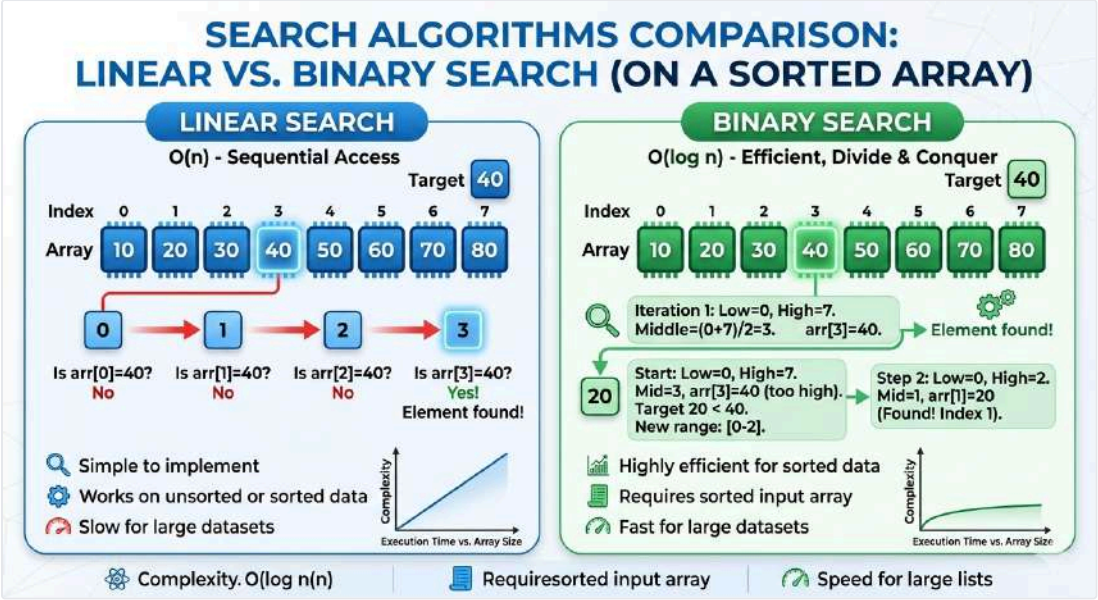
Big O Cheat Sheet with Real-World Analogies

BIG O	NAME	EXAMPLE	REAL-WORLD ANALOGY
O(1)	Constant	Array access by index	Looking up a word in a dictionary by page number
O(log n)	Logarithmic	Binary search	Finding a name in a phone book by halving
O(n)	Linear	Linear search, traversal	Reading every page of a book to find a quote
O(n log n)	Linearithmic	Merge sort, quick sort (avg)	Sorting a deck of cards with merge sort strategy
O(n²)	Quadratic	Bubble sort, nested loops	Comparing every student with every other student
O(2ⁿ)	Exponential	Recursive Fibonacci, subsets	Trying every combination on a lock
O(n!)	Factorial	Permutations, TSP brute force	Arranging n people in all possible seat orders

JAVA — Demonstrating Different Time Complexities

```java
public class ComplexityExamples {

    // O(1) - Constant: Does not depend on input size
    public static int getFirst(int[] arr) {
        return arr[0];
    }

    // O(log n) - Logarithmic: Halves the search space each step
    public static int binarySearch(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) return mid;
            else if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }

    // O(n) - Linear: Visits each element once
    public static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) return i;
        }
        return -1;
    }

    // O(n^2) - Quadratic: Nested iteration over same array
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
```

```
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```



UNDERSTANDING BIG O NOTATION

Big O notation describes the upper bound of an algorithm's growth rate — the worst-case scenario. It strips away constants and lower-order terms to focus on what matters most as input grows. O(3n + 5) simplifies to O(n). O(n^2 + n) simplifies to O(n^2). Why? Because when n is 1 million, the n^2 term (1 trillion) dwarfs the n term (1 million). Think of it like estimating travel time: if your flight is 10 hours and the taxi to the airport is 20 minutes, you say the trip takes about 10 hours. The taxi time is insignificant at scale.

SPACE COMPLEXITY EXPLAINED

Space complexity measures the additional memory an algorithm uses beyond the input itself. If you create a new array of size n to store sorted results, that is O(n) extra space. If you sort in-place by swapping elements, that is O(1) extra space. Real scenario: on a mobile phone with limited RAM, an in-place sorting algorithm is preferred over one that creates copies. Recursive algorithms use O(d) space where d is the recursion depth, because each call adds a frame to the call stack. A recursive function calling itself n times uses O(n) stack space, which can cause StackOverflowError for large inputs.

ANALYZING LOOPS AND NESTED LOOPS

The key to analyzing time complexity is counting how many times the innermost operation executes. A single for-loop from 0 to n is O(n). Two separate for-loops (not nested) are O(n) + O(n) = O(2n) = O(n). A nested loop where both iterate n times gives O(n) × O(n) = O(n^2). But be careful: not all nested loops are O(n^2). If the inner loop runs a constant number of times (say, 5), it is O(5n) = O(n). If the inner loop variable depends on the outer (like j = i to n), the total is roughly n(n-1)/2 which is still O(n^2). Always trace through the actual execution count.

JAVA — Tracing Loop Complexity

```
// Example 1: O(n) — Two separate loops
for (int i = 0; i < n; i++) { /* O(n) */ }
for (int j = 0; j < n; j++) { /* O(n) */ }
// Total: O(n) + O(n) = O(n)

// Example 2: O(n^2) — Nested loops
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) { /* O(n^2) */ }
}

// Example 3: O(n log n) — Outer O(n), inner halves each time
for (int i = 0; i < n; i++) {
    int j = n;
    while (j > 0) {
        j = j / 2;  // Runs log(n) times
    }
```

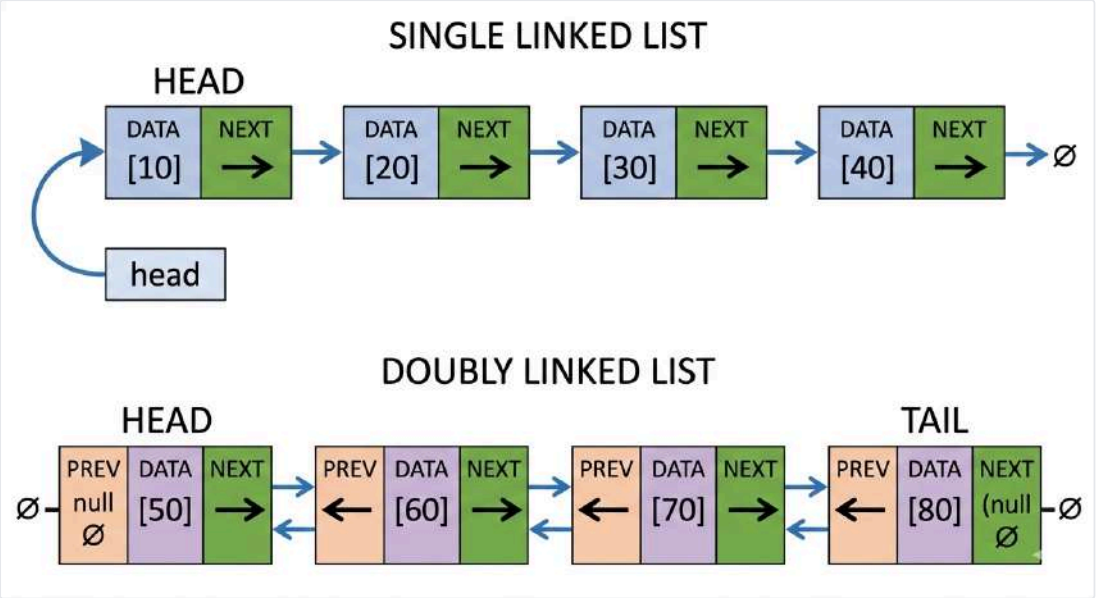
```
}

// Example 4: O(log n) — Loop variable doubles
for (int i = 1; i < n; i *= 2) { /* log(n) iterations */ }
```

BEST, AVERAGE, AND WORST CASE ANALYSIS

Algorithms can perform differently depending on the input. Linear search for an element in an array has a best case of O(1) (element is first), average case of O(n/2) = O(n), and worst case of O(n) (element is last or absent). Quick sort has an average case of O(n log n) but a worst case of O(n^2) when the pivot is always the smallest or largest element. In practice, we usually focus on worst-case analysis because it gives a guarantee: your code will never be slower than this. However, average-case matters for algorithms like quick sort where the worst case is rare with random pivots.

LINKED LISTS: DYNAMIC AND FLEXIBLE DATA STORAGE

A linked list is a linear data structure where each element (node) contains data and a reference (pointer) to the next node. Unlike arrays, linked lists do not require contiguous memory, making insertions and deletions efficient. They are the foundation for implementing stacks, queues, hash table chaining, and adjacency lists in graphs.



Singly Linked List Structure

Array vs Linked List Comparison

FEATURE	ARRAY	LINKED LIST
Memory Layout	Contiguous	Non-contiguous (scattered)
Access by Index	O(1)	O(n)
Insert at Beginning	O(n)	O(1)
Insert at End	O(1) amortized	O(1) with tail pointer
Delete from Middle	O(n)	O(1) if node reference known
Memory Overhead	None (just data)	Extra pointer per node
Cache Performance	Excellent (locality)	Poor (scattered memory)
Size	Fixed or amortized resize	Truly dynamic

JAVA — Singly Linked List Implementation with Insert, Delete, and Traversal

```
// Scenario: Music playlist manager — songs in a linked list
class Node {
    String song;
    Node next;
    Node(String song) {
        this.song = song;
        this.next = null;
    }
}
```

```
class Playlist {
    Node head;

    // Insert at beginning — O(1)
    void addToFront(String song) {
        Node newNode = new Node(song);
        newNode.next = head;
        head = newNode;
    }

    // Insert at end — O(n)
    void addToEnd(String song) {
        Node newNode = new Node(song);
        if (head == null) { head = newNode; return; }
        Node curr = head;
        while (curr.next != null) curr = curr.next;
        curr.next = newNode;
    }

    // Delete by song name — O(n)
    void remove(String song) {
        if (head == null) return;
        if (head.song.equals(song)) { head = head.next; return; }
        Node curr = head;
        while (curr.next != null && !curr.next.song.equals(song)) {
            curr = curr.next;
        }
        if (curr.next != null) curr.next = curr.next.next;
    }

    // Traverse and print — O(n)
    void display() {
        Node curr = head;
        while (curr != null) {
            System.out.print(curr.song + " -> ");
            curr = curr.next;
        }
        System.out.println("END");
    }

    public static void main(String[] args) {
        Playlist pl = new Playlist();
        pl.addToEnd("Shape of You");
        pl.addToEnd("Blinding Lights");
        pl.addToFront("Bohemian Rhapsody");
        pl.display();
        // Bohemian Rhapsody -> Shape of You -> Blinding Lights -> END
        pl.remove("Shape of You");
        pl.display();
        // Bohemian Rhapsody -> Blinding Lights -> END
    }
}
```

SINGLY VS DOUBLY VS CIRCULAR LINKED LISTS

A singly linked list has nodes with data and a 'next' pointer — traversal is one-directional. A doubly linked list adds a 'prev' pointer, allowing traversal in both directions at the cost of extra memory per node. Real scenario: a web browser's back/forward navigation uses a doubly linked list where each page is a node with prev and next pointers. A circular linked list connects the last node back to the first, forming a ring. This is used in round-robin CPU scheduling where processes take turns in a cycle, and in multiplayer board games where turns rotate continuously.

REVERSING A LINKED LIST

Reversing a linked list is one of the most asked interview questions and tests your understanding of pointer manipulation. The iterative approach uses three pointers: prev (starts null), curr (starts at head), and next (temporary). For each node, you save the next reference, point curr.next to prev, advance prev to curr, and advance curr to next. After the loop, prev is the new head. This runs in O(n) time and O(1) space. The recursive approach reverses the rest of the list first, then fixes the current node's pointer, using O(n) stack space.

JAVA — Iterative and Recursive Linked List Reversal

```
// Iterative reversal — O(n) time, O(1) space
Node reverseIterative(Node head) {
    Node prev = null, curr = head, next = null;
    while (curr != null) {
        next = curr.next;   // Save next
```

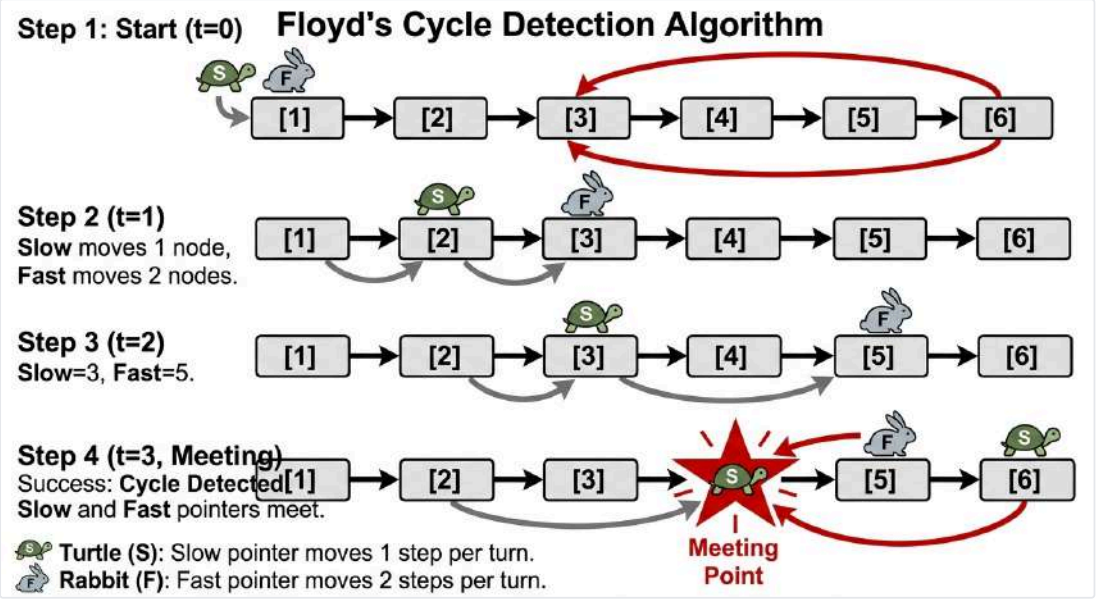
```
        curr.next = prev;   // Reverse pointer
        prev = curr;        // Move prev forward
        curr = next;        // Move curr forward
    }
    return prev; // New head
}

// Recursive reversal — O(n) time, O(n) space (call stack)
Node reverseRecursive(Node head) {
    if (head == null || head.next == null) return head;
    Node newHead = reverseRecursive(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
```

DETECTING CYCLES: FLOYD'S TORTOISE AND HARE ALGORITHM

A cycle in a linked list occurs when a node's next pointer references a previous node, creating an infinite loop. Floyd's algorithm uses two pointers: a slow pointer (moves 1 step) and a fast pointer (moves 2 steps). If there is a cycle, the fast pointer will eventually meet the slow pointer inside the loop, like two runners on a circular track where the faster one laps the slower one. If the fast pointer reaches null, there is no cycle. This runs in O(n) time and O(1) space. Real scenario: detecting infinite redirect loops in URL shortener services where links can accidentally point to each other.

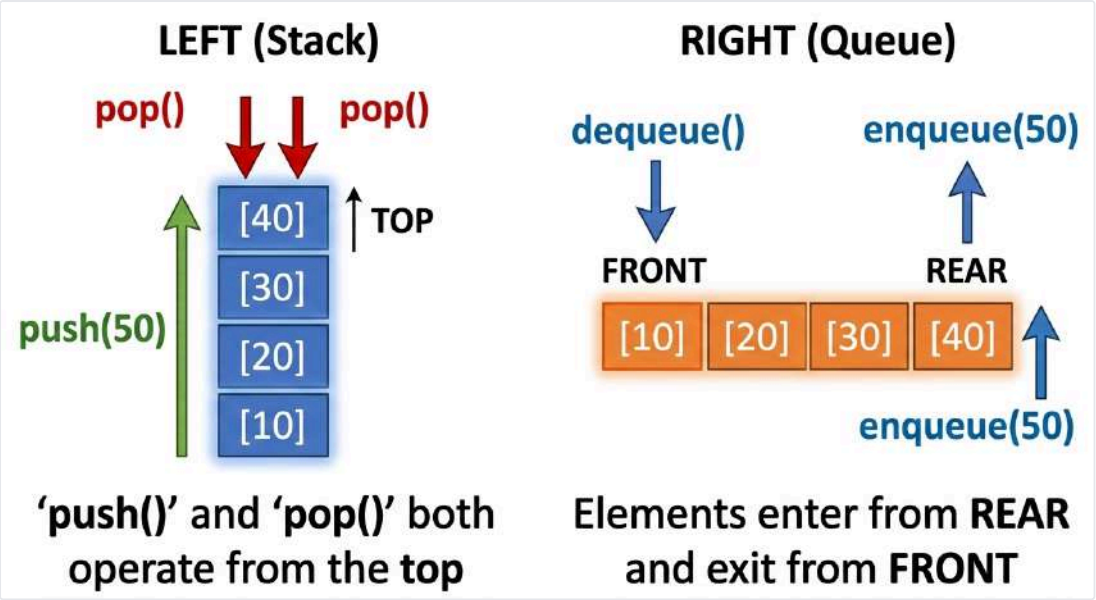
JAVA — Floyd's Cycle Detection Algorithm

```
public static boolean hasCycle(Node head) {
    if (head == null) return false;
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;        // 1 step
        fast = fast.next.next;   // 2 steps
        if (slow == fast) return true;
    }
    return false;
}
```



STACKS AND QUEUES: LIFO AND FIFO DATA STRUCTURES

Stacks and queues are abstract data types that restrict how elements are added and removed. A stack follows Last-In-First-Out (LIFO) — like a stack of plates where you always take from the top. A queue follows First-In-First-Out (FIFO) — like a line at a bank where the first person in line is served first. Despite their simplicity, they power critical systems from function calls to task scheduling.



Stack (LIFO) vs Queue (FIFO) Operations

Stack and Queue Operations Complexity

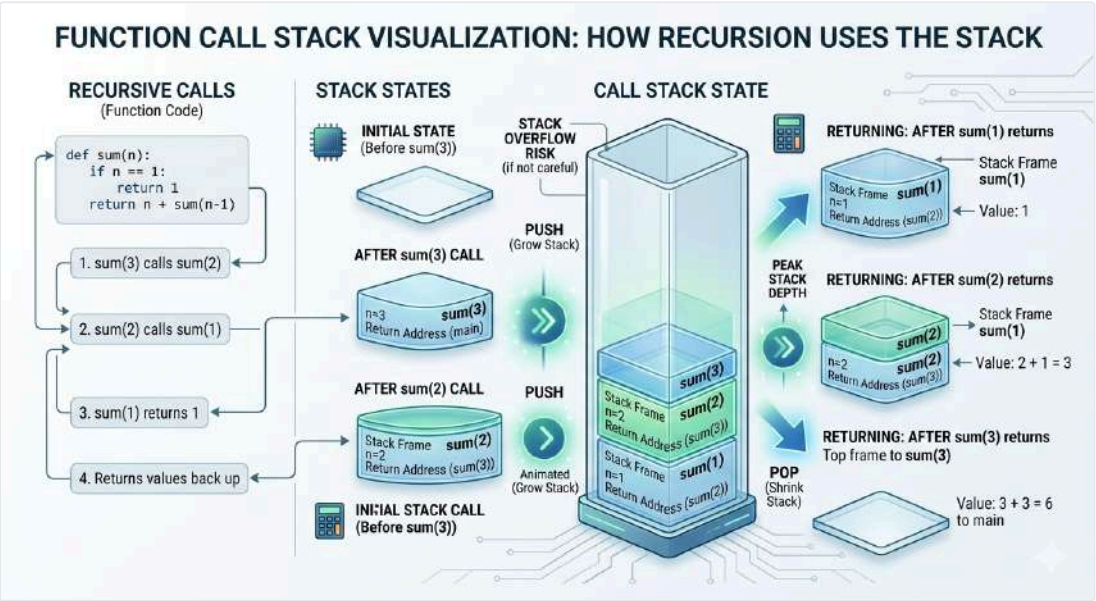
OPERATION	STACK	QUEUE
Insert	push() — O(1)	enqueue() — O(1)
Remove	pop() — O(1)	dequeue() — O(1)
Peek/Front	peek() — O(1)	front() — O(1)
Search	O(n)	O(n)
IsEmpty	O(1)	O(1)

JAVA Stack Implementation Using Array and Classic Interview Problem: Balanced Parentheses

```java
import java.util.Stack;

public class StackProblems {

    // Problem: Check if parentheses are balanced
    // Scenario: Code editor validating brackets in source code
    public static boolean isBalanced(String expr) {
        Stack<Character> stack = new Stack<>();
        for (char ch : expr.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            } else if (ch == ')' || ch == '}' || ch == ']') {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '[')) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(isBalanced("{[()]}")); // true
        System.out.println(isBalanced("{[(])}")); // false
        System.out.println(isBalanced("(())")); // true
        System.out.println(isBalanced("(]")); // false
    }
}
```

JAVA Queue Implementation Using LinkedList and BFS Level-Order Scenario

```java
import java.util.LinkedList;
import java.util.Queue;

public class QueueDemo {

    // Scenario: Customer service ticket system
    public static void ticketSystem() {
        Queue<String> tickets = new LinkedList<>();

        // Customers submit tickets (enqueue)
        tickets.add("Ticket#101 - Login issue");
        tickets.add("Ticket#102 - Payment failed");
        tickets.add("Ticket#103 - Order tracking");

        // Support agents process tickets (dequeue) — FIFO
        while (!tickets.isEmpty()) {
            String current = tickets.poll();
            System.out.println("Processing: " + current);
        }
        // Output:
        // Processing: Ticket#101 - Login issue
        // Processing: Ticket#102 - Payment failed
        // Processing: Ticket#103 - Order tracking
    }

    public static void main(String[] args) {
        ticketSystem();
    }
}
```



STACK: REAL-WORLD APPLICATIONS

Stacks are used in many systems you interact with daily. The Undo/Redo feature in text editors pushes each action onto a stack; pressing Ctrl+Z pops the last action and reverses it. Web browser back buttons maintain a stack of visited URLs — each new page is pushed, and clicking back pops the top. The Java Virtual Machine uses a call stack to manage method invocations: each method call pushes a stack frame containing local variables and return address, and returning from the method pops the frame. Expression evaluation in calculators uses stacks to convert infix to postfix and evaluate the result.

QUEUE: REAL-WORLD APPLICATIONS

Queues model fairness: first come, first served. Operating systems use queues for CPU task scheduling — processes waiting for the CPU are placed in a ready queue and served in order. Print spoolers queue print jobs so multiple users can share a single printer. Message queues like RabbitMQ and Apache Kafka buffer messages between microservices to handle traffic spikes. In Breadth-First Search (BFS), a queue processes graph or tree nodes level by level. Even your Netflix streaming buffer uses a queue: video chunks are downloaded and queued for playback in order.

IMPLEMENTING A STACK USING QUEUES

A classic interview problem asks you to implement a stack's LIFO behavior using only queue operations. The trick is: when you push an element, enqueue it, then dequeue and re-enqueue all previous elements so the newest element is now at the front. This makes push O(n) but pop O(1). Alternatively, you can make push O(1) and pop O(n) by transferring elements during pop. This problem tests your understanding of how these data structures relate and can simulate each other. In practice, you would never do this in production, but it demonstrates the conceptual relationship between LIFO and FIFO.

JAVA — Stack Implementation Using Two Queues

```java
import java.util.LinkedList;
import java.util.Queue;

class StackUsingQueues {
    Queue<Integer> q1 = new LinkedList<>();
    Queue<Integer> q2 = new LinkedList<>();

    // Push: O(n) — move all to q2, add new to q1, move back
    void push(int x) {
        q2.add(x);
        while (!q1.isEmpty()) q2.add(q1.poll());
        Queue<Integer> temp = q1;
        q1 = q2;
        q2 = temp;
    }

    // Pop: O(1)
    int pop() {
        return q1.poll();
    }

    // Peek: O(1)
    int top() {
        return q1.peek();
    }
}
```

DEQUE: THE DOUBLE-ENDED QUEUE

A deque (pronounced 'deck') allows insertion and removal from both the front and the rear in O(1) time. It can function as both a stack and a queue depending on which end you use. Java provides ArrayDeque, which is faster than both Stack and LinkedList for stack and queue operations. Real-world use case: in a navigation app, a deque can manage a list of recently visited locations — new visits are added to the front, old entries are removed from the rear, and the user can navigate both forward and backward. The sliding window maximum problem uses a deque to maintain the maximum element in the current window efficiently in O(n) time.

JAVA — Sliding Window Maximum Using Deque

```java
import java.util.ArrayDeque;
import java.util.Deque;

public class SlidingWindowMax {
    public static int[] maxSlidingWindow(int[] nums, int k) {
        Deque<Integer> deque = new ArrayDeque<>(); // stores indices
        int[] result = new int[nums.length - k + 1];

        for (int i = 0; i < nums.length; i++) {
            // Remove indices outside the window
            while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
                deque.pollFirst();
            }
            // Remove smaller elements from the back
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
                deque.pollLast();
            }
            deque.addLast(i);
            // Record result once window is fully formed
            if (i >= k - 1) {
                result[i - k + 1] = nums[deque.peekFirst()];
            }
        }
        return result;
    }
    // Input: nums=[1,3,-1,-3,5,3,6,7], k=3
    // Output: [3,3,5,5,6,7]
}
```

CONCLUSION

This Refcard covered the essential -building blocks of Data Structures and Algorithms that every developer must master. Arrays provide O(1) random access and are ideal for fixed-size, index-heavy workloads. Strings require careful handling due to immutability, and techniques like the sliding window and frequency arrays unlock efficient solutions to common problems. Time and space complexity analysis using Big O notation gives you the vocabulary to evaluate trade-offs and predict performance before your code even runs.

Linked lists offer dynamic memory allocation and O(1) insertions at known positions, making them ideal for scenarios where size is unpredictable. Stacks (LIFO) and queues (FIFO) are deceptively simple structures that underpin function calls, undo systems, BFS traversal, and task scheduling. Understanding when to use each data structure — and more importantly, why — separates a coder from an engineer.

Common beginner mistakes to avoid: (1) Using arrays when frequent insertions or deletions are needed — consider linked lists instead. (2) Concatenating strings in a loop without StringBuilder, causing $O(n^2)$ performance. (3) Confusing time complexity of different operations — array access is O(1) but insertion is O(n). (4) Forgetting to handle edge cases like empty structures, single elements, and null pointers. (5) Not considering space complexity alongside time complexity. Next steps: practice these concepts on LeetCode (Easy to Medium problems), study recursion and sorting algorithms, then advance to trees, graphs, and dynamic programming covered in subsequent Refcards in this series.